Reviewer Pack — Minimal Frobenius Class Dimension and Communication Complexi…

Entry 0a0e5563b772 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 13:01:31 UTC

Minimal Frobenius Class Dimension and Communication Complexity Rank

Entry ID: `0a0e5563b772` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 13:01:31 UTC

1. Conjecture

Field A (mathematical branch): Frobenius Algebras and Representation Theory **Field B** (complexity object): Communication Complexity

Statement:

> For every boolean function f, the dimension of its minimal Frobenius class in the algebraic structure associated with the communication complexity of f is upper-bounded by a polynomial in the number of inputs n, i.e., dim(min_frobenius(f)) = O(n^k) for some constant k.

Rationale (proposer's reasoning):

> Frobenius algebras provide a framework to study noncommutative structures and their representations. The minimal Frobenius class dimension could potentially encode information about the complexity of computing the function, while communication complexity measures the amount of information that must be communicated between parties to compute the function. This conjecture suggests that these two mathematical objects are related in a structured way.

Taxonomy category: `Communication Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f9990bddb720ba94`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean dimension of minimal Frobenius classes, across all tested functions with n inputs, is less than or equal to n^k for some constant k and all seeds produce communication complexity ranks ≤ 10. The conjecture is falsified if any seed produces a mean dimension > n^k or a communication complexity rank > 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal Frobenius class dimension" AND "communication complexity rank" - "Frobenius algebra" AND "dimension of boolean function" - "polynomial bound" AND "algebraic structure communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = int(math.log2(len(f)))
        if len(f) != 2**n:
            raise ValueError("Invalid boolean function length")

        # Simplified version of communication complexity rank calculation
        return sum(1 for x in range(n) if f[x] != f[x + n])

    def frobenius_class_dimension(f):
        n = int(math.log2(len(f)))
        if len(f) != 2**n:
            raise ValueError("Invalid boolean function length")

        # Simplified version of Frobenius class dimension calculation
        return sum(1 for x in range(n) if f[x] == f[x + n])

    results = []
    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
        f = generate_boolean_function(n)
        rank = communication_complexity_rank(f)
```

```
        dim = frobenius_class_dimension(f)

        if rank > 10:
            return {
                "metric_name": "communication_complexity_rank",
                "metric_value": rank,
                "instances_tested": 1,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": f"rank={rank} > 10"
            }

        results.append(dim)

    mean_dim = sum(results) / len(results)
    return {
        "metric_name": "frobenius_class_dimension",
        "metric_value": mean_dim,
        "instances_tested": 30,
        "n_max": max(40, n),
        "conjecture_holds": mean_dim <= n**2,  # Simplified polynomial bound
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [random.randint(1000, 9999) f

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='rank>10' first_failing_seed={first_failing_seed}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means that the pre-registered support conditions could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14156
2	propose	ollama_remote	glm4:latest	0	0	13358
3	preregistration	ollama_remote	glm4:latest	0	0	9543
4	novelty	ollama_remote	glm4:latest	0	0	10027
5	novelty	ollama_remote	glm4:latest	0	0	14580
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13236
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18016
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32859
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17481
10	judge	ollama_remote	glm4:latest	0	0	38978

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 182233 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/0a0e5563b772.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0a0e5563b772.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0a0e5563b772.tar.gz` (if generated)